Checkpoint Artifact Submission — Community Guide

Project: BAIO (Bioinformatics AI for Open-Set Detection) **Course:** Open Source Software — Saint Louis University **Sprint:** 5 **Tech Lead:** Mainuddin **Team:** Mainuddin, Luis Palmejar, Kevin Yang **Repository:** https://github.com/oss-slu/baio

---

Artifact Identification

Checkpoint Catalog Item: Community Guide

Artifact: This document serves as BAIO's official Community Guide, covering how new contributors can join the project, how to set up the development environment, how to submit issues and pull requests, and the norms and expectations for participation in the BAIO open-source community.

Rationale

We selected the Community Guide checkpoint because BAIO is entering a phase where outside contributors — classmates, future OSS students, and bioinformatics researchers — are increasingly likely to encounter the project. Our Sprint 4 metrics report revealed that the team's PR authorship was heavily concentrated (the Tech Lead authored 77% of merged PRs), and seven open issues had no assigned owner. A clear Community Guide directly addresses these structural gaps by lowering the barrier for new contributors to pick up unowned issues, submit well-formed pull requests, and understand the project's standards without needing to ask.

This checkpoint also contributes directly to our final product and community strategy. BAIO's long-term goal is to be a usable, extensible tool for metagenomic analysis — not just a course deliverable. For that to happen, the project must be approachable to people outside the original team. The Community Guide establishes the foundation for sustainable open-source participation: it documents our workflow, tooling, contribution standards, and communication norms in one place. Completing this now, while the core architecture is stable, ensures the guide reflects real project conventions rather than aspirational ones.

BAIO Community Guide

Welcome

BAIO is an open-source metagenomic analysis platform that uses machine learning to classify DNA sequences and flag novel organisms. We welcome contributions from developers, bioinformaticians, students, and researchers. Whether you are fixing a bug, adding a feature, improving documentation, or writing tests — your work matters.

Table of Contents

1. Code of Conduct
2. Who We Are

3. Ways to Contribute
4. Development Setup
5. Project Structure
6. Branching and Workflow
7. Opening Issues
8. Submitting Pull Requests
9. Code Standards
10. Testing
11. Getting Help

Code of Conduct

All participants in the BAIO project are expected to:

- Be respectful and constructive in all communication (issues, PRs, comments).
- Welcome and support contributors of all experience levels.
- Give credit where it is due — acknowledge others' contributions.
- Assume good intent before assuming malice in ambiguous situations.
- Raise concerns privately with the Tech Lead before escalating publicly.

Harassment, discrimination, or deliberately hostile behavior of any kind will result in removal from the project.

Who We Are

Role	Name	GitHub
Tech Lead	Mainuddin Sarker	@mainuddinMains
Developer	Kevin Yang	@TommySailami
Developer	Luis Palmejar	@LuisPalmejar21

The Tech Lead is responsible for reviewing and merging pull requests, maintaining CI, and resolving architectural questions. Developers own features and bug fixes. All team members are expected to review each other's PRs.

Ways to Contribute

Good first contributions:

- Fix a bug listed in our open issues tagged `good first issue`
- Improve or expand the documentation
- Add or fix a test in `tests/`
- Report a bug you discovered while using the app

Intermediate contributions:

- Implement an open feature issue (check for an existing issue before starting)

- Improve classifier accuracy or add a new model type
- Add frontend components or improve the results dashboard
- Improve the AI assistant prompts

Advanced contributions:

- Complete Evo2 inference pipeline integration (issue #179)
- Database-backed session persistence (issue #162)
- Google OAuth authentication (issue #182)

Before starting significant work, **comment on the issue** to let the team know you are working on it. This prevents duplicate effort.

Development Setup

Prerequisites

Tool	Version	Notes
Python	3.10+	Via Conda recommended
Node.js	18+	For the frontend
Git	Any recent	Required
Docker Desktop	Any recent	Optional — simplest setup

Option A: Docker (Recommended for new contributors)

```
# 1. Clone the repo
git clone https://github.com/oss-slu/baio.git
cd baio

# 2. Copy and configure environment variables
cp .env.example .env
# Open .env and add your API keys (optional — app works without them)

# 3. Build and run
docker compose up --build
```

Open http://localhost:4173 in your browser. The API docs are at http://localhost:8080/docs.

Option B: Local Development (Mac)

```
# 1. Clone and enter the repo
git clone https://github.com/oss-slu/baio.git
cd baio
```

```
# 2. Create and activate the Python environment
conda env create -f environment.yml
conda activate baio

# 3. Start the backend (Terminal 1)
uvicorn backend.app.main:app --reload --port 8080

# 4. Start the frontend (Terminal 2)
cd frontend
npm install
npm run dev
```

Open http://localhost:5173 in your browser.

Environment Variables

Copy `.env.example` to `.env`. The app works without API keys, but the AI chatbot will use mock responses.

```
OPENROUTER_API_KEY=optional
GEMINI_API_KEY=optional
```

Project Structure

```
baio/
├── api/                    # FastAPI backend (endpoints, LLM client)
├── backend/                # Application logic
├── binary_classifiers/     # ML classification core (SVM, RandomForest)
├── frontend/               # React + TypeScript + Vite UI
│   └── src/
│       ├── components/     # UI components (Header, Input, Results, etc.)
│       ├── App.tsx         # Root component
│       └── api.ts          # Backend API calls
├── prompting/              # LLM prompt framework and techniques
├── tests/                  # Pytest test suite
├── data/                   # Sample FASTA files for testing
├── scripts/                # Evaluation and utility scripts
├── docs/                   # Project documentation
├── docker-compose.yml      # Docker setup
├── pyproject.toml          # Python dependencies (source of truth)
└── environment.yml         # Conda environment
```

Key principle: All Python dependencies live in `pyproject.toml`. Do not add packages directly to the `Dockerfile`.

Branching and Workflow

We use a **feature branch workflow**:

```
main  ←  feat/your-feature-name
         fix/your-bug-description
         docs/what-you-documented
         test/what-you-tested
```

Branch naming conventions:

Type	Prefix	Example
New feature	`feat/`	`feat/evo2-inference`
Bug fix	`fix/`	`fix/duplicate-chat-widget`
Documentation	`docs/`	`docs/api-reference`
Tests	`test/`	`test/classify-endpoint`
Refactor	`refactor/`	`refactor/classifier-module`

Never push directly to `main`. All changes must go through a pull request.

Workflow steps:

```
# 1. Create your branch from main
git checkout main
git pull origin main
git checkout -b feat/your-feature-name

# 2. Make your changes, commit with clear messages
git add <specific files>
git commit -m "feat: add Evo2 confidence scoring to classify endpoint"

# 3. Push and open a PR
git push origin feat/your-feature-name
```

Then open a pull request on GitHub.

Opening Issues

Before opening an issue, search existing issues to avoid duplicates.

Use the provided templates:

- **Feature / Enhancement** — for new functionality or improvements
- **Epic** — for large, multi-part initiatives

A good issue includes:

- A clear, specific title
- What you expected vs. what happened (for bugs)
- Steps to reproduce (for bugs)
- Acceptance criteria — what "done" looks like
- Your environment (OS, Python version, Node version) if relevant

Label your issue appropriately: `bug`, `enhancement`, `documentation`, `good first issue`.

Submitting Pull Requests

Use the pull request template. Every PR must include:

1. **Fixes #issue_number** at the top — links the PR to an issue
2. **What was changed** — which files and components
3. **Why it was changed** — the problem being solved
4. **How it was changed** — key implementation details
5. **Screenshots** — for any UI changes

PR checklist before requesting review:

- ☐ Tests pass locally (`pytest tests/`)
- ☐ No new linting errors
- ☐ Frontend builds without errors (`npm run build` in `frontend/`)
- ☐ Docker build succeeds (`docker compose up --build`)
- ☐ Branch is up to date with `main`
- ☐ PR description is complete

Review expectations:

- Every PR requires at least **one approval** before merge
- Do not merge your own PR without review unless the Tech Lead explicitly approves
- Address all review comments or explain why you disagree — do not silently close them
- PRs should not be merged the same day they are opened (allow time for review)

Code Standards

Python

- Follow PEP 8 style conventions
- Use type hints for function signatures
- Keep functions focused — one responsibility per function
- Add docstrings to public functions and classes
- All new dependencies go in `pyproject.toml`, not the `Dockerfile`

TypeScript / React

- Use TypeScript types; avoid `any`

- Keep components small and focused
- Co-locate component logic with its component file
- Use the existing `api.ts` for all backend calls — do not fetch directly from components

Git Commits

Write commit messages in the imperative mood with a type prefix:

```
feat: add GC content heatmap to results dashboard
fix: resolve duplicate chat widget in DOM
docs: update README with Docker setup instructions
test: add unit tests for classify endpoint
refactor: extract k-mer logic into separate module
```

A good commit message answers: *what does this commit do, and why?*

Testing

```
# Activate environment
conda activate baio

# Run all tests
pytest tests/

# Run a specific test file
pytest tests/test_api_classification.py

# Run with coverage report
pytest --cov=. tests/
```

When to write tests:

- Every new API endpoint must have at least one test
- Every bug fix should have a regression test that would have caught it
- New ML model integrations require at least a smoke test

We use `pytest`. Tests live in the `tests/` directory. Keep test files named `test_<module>.py`.

Getting Help

- **Found a bug?** Open a GitHub issue using the issue template
- **Have a question about the codebase?** Comment on the relevant issue or PR
- **Need architectural guidance?** Tag `@mainuddinMains` in your issue or PR
- **Not sure where to start?** Look for issues labeled `good first issue`

We are a small team and response times may vary, but we aim to respond to all issues and PRs within a few days.

BAIO Community Guide — Sprint 5 Checkpoint Artifact Prepared by Mainuddin Sarker, Tech Lead Saint Louis University — Open Source Software, April 2026